Solution: Happy Number

Problem Statement

Any number will be called a happy number if, after repeatedly replacing it with a number equal to the **sum of the square of all of its digits, leads us to the number `1`. All other (not-happy) numbers will never reach `1`**. Instead, they will be stuck in a cycle of numbers that does not include `1`.

Given a positive number `n`, return `true` if it is a happy number otherwise return `false`.

Example 1:

> **Input:** 23
>
> **Output:** true (23 is a happy number)
>
> **Explanations:** Here are the steps to find out that 23 is a happy number:
>
> $2^2 + 3^2 = 4 + 9 = 13$
> $1^2 + 3^2 = 1 + 9 = 10$
> $1^2 + 0^2 = 1 + 0 = 1$

Example 2:

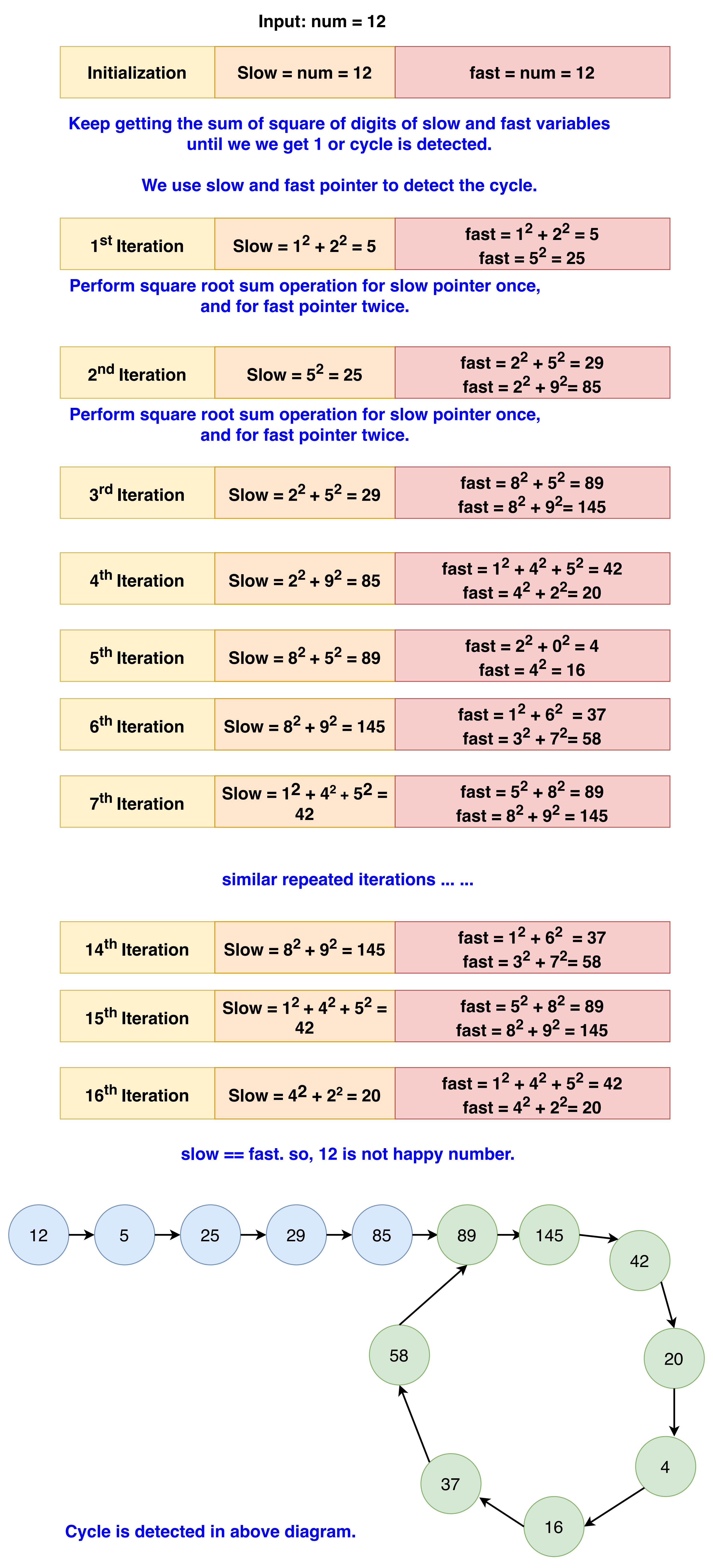
> **Input:** 12
>
> **Output:** false (12 is not a happy number)
>
> **Explanations:** Here are the steps to find out that 12 is not a happy number:
>
> $1^2 + 2^2 = 1 + 4 = 5$
> $5^2 = 25$
> $2^2 + 5^2 = 4 + 25 = 29$
> $2^2 + 9^2 = 4 + 81 = 85$
> $8^2 + 5^2 = 64 + 25 = 89$
> $8^2 + 9^2 = 64 + 81 = 145$
> $1^2 + 4^2 + 5^2 = 1 + 16 + 25 = 42$
> $4^2 + 2^2 = 16 + 4 = 20$
> $2^2 + 0^2 = 4 + 0 = 4$
> $4^2 = 16$
> $1^2 + 6^2 = 1 + 36 = 37$
> $3^2 + 7^2 = 9 + 49 = 58$
> $5^2 + 8^2 = 25 + 64 = 89$
>
> Please note the cycle from 89 -> 89.

Solution

The process, defined above, to find out if a number is a happy number or not, always ends in a cycle. If the number is a happy number, the process will be stuck in a cycle on number ‘1,’ and if the number is not a happy number then the process will be stuck in a cycle with a set of numbers. As we saw in Example-2 while determining if ‘12’ is a happy number or not, our process will get stuck in a cycle with the following numbers: 89 -> 145 -> 42 -> 20 -> 4 -> 16 -> 37 -> 58 -> 89

We saw in the LinkedList Cycle problem that we can use the **Fast & Slow pointers** method to find a cycle among a set of elements. As we have described above, each number will definitely have a cycle. Therefore, we will use the same fast & slow pointer strategy to find the cycle and once the cycle is found, we will see if the cycle is stuck on number ‘1’ to find out if the number is happy or not.

Here is the visual representation of this algorithm for Example-2:



Input: num = 12

Initialization	Slow = num = 12	fast = num = 12

Keep getting the sum of square of digits of slow and fast variables until we we get 1 or cycle is detected.

We use slow and fast pointer to detect the cycle.

1^{st} Iteration	Slow = $1^2 + 2^2 = 5$	fast = $1^2 + 2^2 = 5$ fast = $5^2 = 25$

Perform square root sum operation for slow pointer once, and for fast pointer twice.

2^{nd} Iteration	Slow = $5^2 = 25$	fast = $2^2 + 5^2 = 29$ fast = $2^2 + 9^2 = 85$

Perform square root sum operation for slow pointer once, and for fast pointer twice.

3^{rd} Iteration	Slow = $2^2 + 5^2 = 29$	fast = $8^2 + 5^2 = 89$ fast = $8^2 + 9^2 = 145$
4^{th} Iteration	Slow = $2^2 + 9^2 = 85$	fast = $1^2 + 4^2 + 5^2 = 42$ fast = $4^2 + 2^2 = 20$
5^{th} Iteration	Slow = $8^2 + 5^2 = 89$	fast = $2^2 + 0^2 = 4$ fast = $4^2 = 16$
6^{th} Iteration	Slow = $8^2 + 9^2 = 145$	fast = $1^2 + 6^2 = 37$ fast = $3^2 + 7^2 = 58$
7^{th} Iteration	Slow = $1^2 + 4^2 + 5^2 = 42$	fast = $5^2 + 8^2 = 89$ fast = $8^2 + 9^2 = 145$

similar repeated iterations ... ...

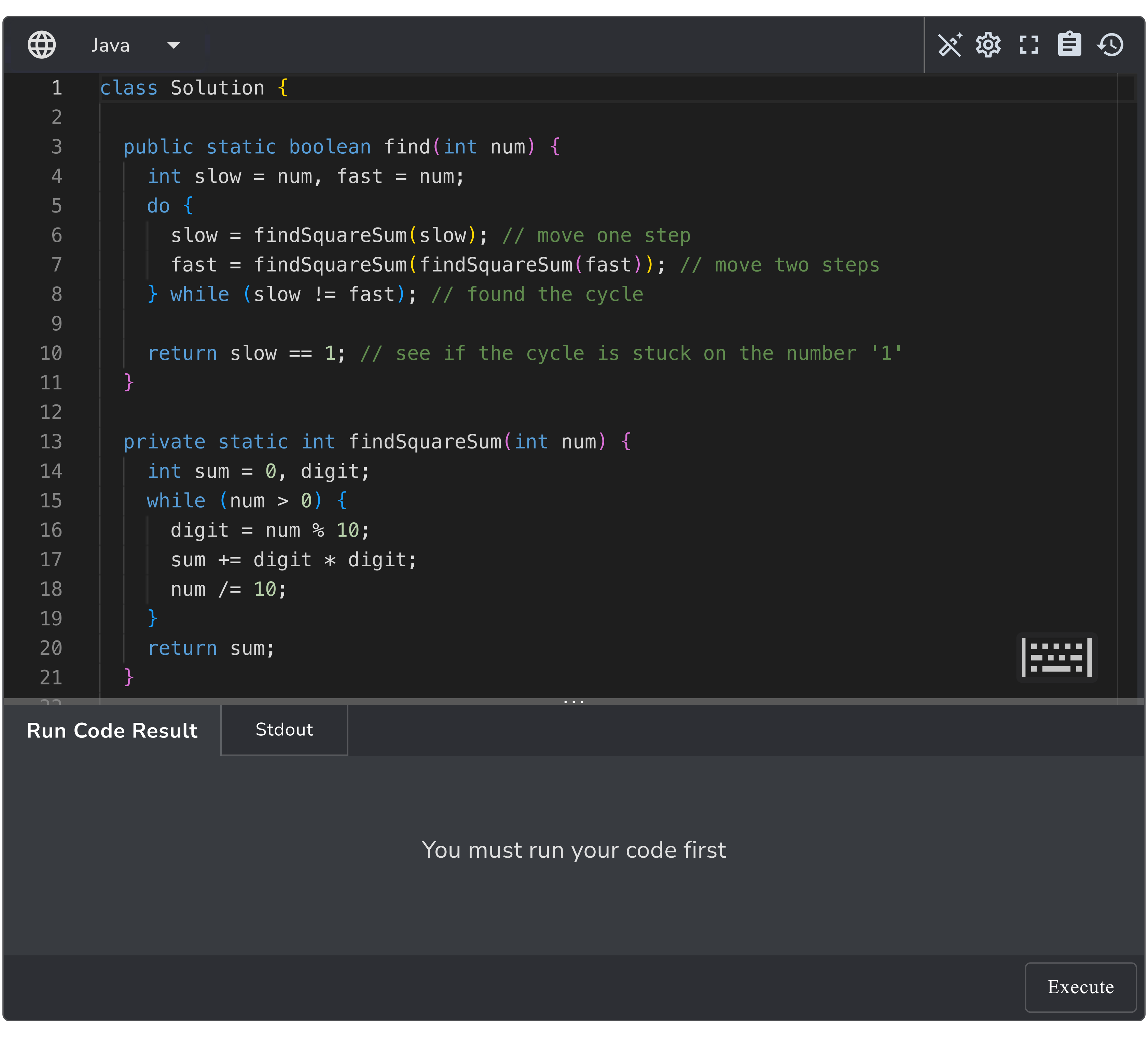
14^{th} Iteration	Slow = $8^2 + 9^2 = 145$	fast = $1^2 + 6^2 = 37$ fast = $3^2 + 7^2 = 58$
15^{th} Iteration	Slow = $1^2 + 4^2 + 5^2 = 42$	fast = $5^2 + 8^2 = 89$ fast = $8^2 + 9^2 = 145$
16^{th} Iteration	Slow = $4^2 + 2^2 = 20$	fast = $1^2 + 4^2 + 5^2 = 42$ fast = $4^2 + 2^2 = 20$

slow == fast. so, 12 is not happy number.

Cycle is detected in above diagram.

Code

Here is what our algorithm will look like:



```java
class Solution {

  public static boolean find(int num) {
    int slow = num, fast = num;
    do {
      slow = findSquareSum(slow); // move one step
      fast = findSquareSum(findSquareSum(fast)); // move two steps
    } while (slow != fast); // found the cycle

    return slow == 1; // see if the cycle is stuck on the number '1'
  }

  private static int findSquareSum(int num) {
    int sum = 0, digit;
    while (num > 0) {
      digit = num % 10;
      sum += digit * digit;
      num /= 10;
    }
    return sum;
  }
```

```
You must run your code first
```

Time Complexity

The time complexity of the algorithm is difficult to determine. However we know the fact that all unhappy numbers eventually get stuck in the cycle: 4 -> 16 -> 37 -> 58 -> 89 -> 145 -> 42 -> 20 -> 4

This sequence behavior tells us two things:

1. If the number N is less than or equal to 1000, then we reach the cycle or ‘1’ in at most 1001 steps.
2. For N > 1000N>1000, suppose the number has ‘M’ digits and the next number is ‘N1’. From this, we know that the sum of the squares of the digits of ‘N’ is at most 9^2M, or $81M$ (this will happen when all digits of ‘N’ are ‘9’).

This means:

1. N1 < 81M.
2. As we know M = log(N+1).
3. Therefore: N1 < 81 * log(N+1) => N1 = O(logN).
4. This concludes that the above algorithm will have a time complexity of O(logN).

Space Complexity

The algorithm runs in constant space O(1).

← Previous
Happy Number (Medium)

Next →
Problem Challenge 1:
Palindrome LinkedList (Medium)

☐ Mark as Completed